Assignment #3 Fourier Transform

測量115 F64111134 施淳譯

1. Implement Fourier Transform and Inverse Fourier Transform

```python
# === Apply 2D Fourier Transform ===
f1 = np.fft.fft2(img1)
fshift1 = np.fft.fftshift(f1)
f2 = np.fft.fft2(img2)
fshift2 = np.fft.fftshift(f2)

# === Compute magnitude ===
magnitude_spectrum1 = 20 * np.log(np.abs(fshift1) + 1) # use +1 to avoid log 0 and scale 20 for visualization
magnitude_spectrum2 = 20 * np.log(np.abs(fshift2) + 1)

# === Inverse Fourier Transform ===
f_ishift1 = np.fft.ifftshift(fshift1) # shift back
img_reconstructed1 = np.fft.ifft2(f_ishift1) # inverse FFT
img_reconstructed1 = np.abs(img_reconstructed1)
f_ishift2 = np.fft.ifftshift(fshift2)
img_reconstructed2 = np.fft.ifft2(f_ishift2)
img_reconstructed2 = np.abs(img_reconstructed2)
```

Figure 1. Code of Fourier Transform and Inverse Fourier Transform

- `np.fft.fft2` computes the 2-dimensional Discrete Fourier Transform.
- `np.fft.ffshift` shifts the zero-frequency component to the center of the spectrum.
- `log(abs(fshift) + 1) * 20` is used to visualize the frequency spectrum. The logarithm enhances the visibility of differences between low and high frequencies ;adding 1 prevents taking the logarithm of 0, and multiplying 20 serves as a scaling factor to expand the magnitude.
- `np.fft.ifftshift` shifts the frequency coordinates back to their original positions.
- `np.fft.ifft2` computes the 2-dimensional Inverse Discrete Fourier Transform.

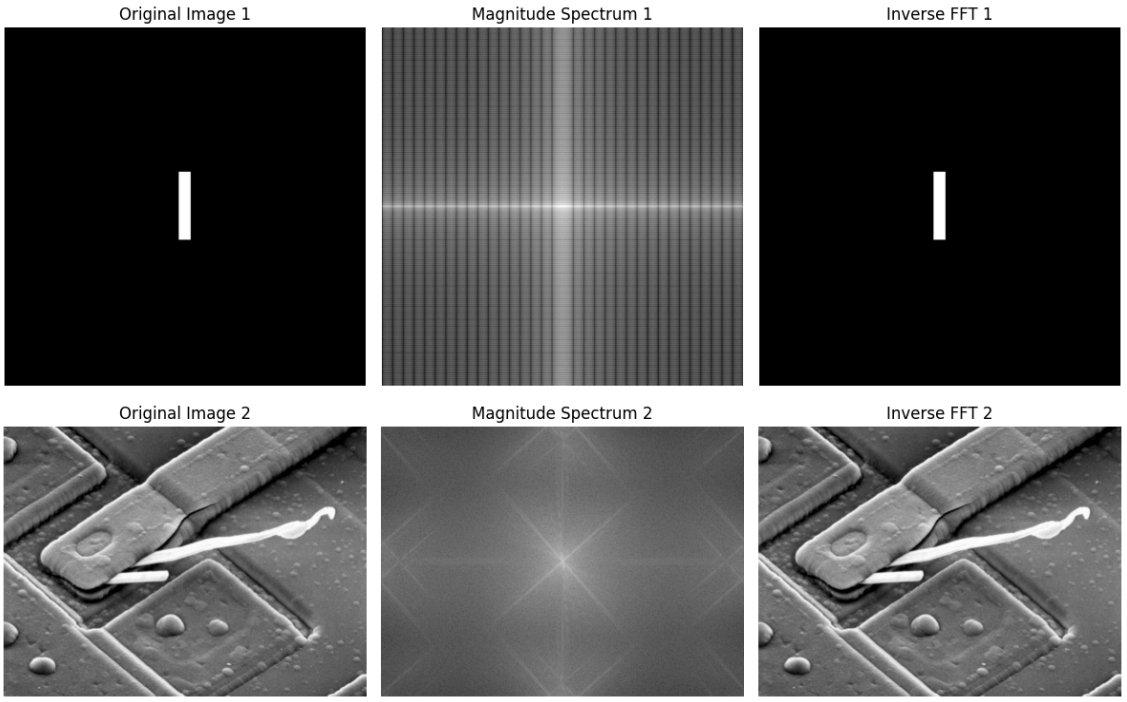


Figure 2. Result of Fourier Transform and Inverse Fourier Transform

From Figure 2, several observations can be made. In Image 1, the original image shows a vertical white rectangle in the center. After applying the Fourier Transform, we obtain its magnitude spectrum. The low-frequency components are concentrated in the center of the spectrum. We can see bright bands in both the horizontal and vertical directions, indicating that the image contains intensity variations (edges) in both directions. The horizontal bright band corresponds to vertical edges in the spatial domain, while the vertical bright band corresponds to horizontal edges. Since the horizontal bright band is much stronger, it implies that the image has stronger vertical edges — consistent with the original image where the vertical rectangle produces prominent vertical boundaries.

In Image 2, the magnitude spectrum shows bright lines along the vertical axis and along ±45° directions, meaning there are significant intensity variations along these orientations. This matches the original image, which contains many diagonal (≈45°) edges and textures.

Finally, in the third column of both Image 1 and Image 2, the results of the Inverse Fourier Transform are shown. The reconstructed images are identical to the originals, confirming that the Fourier Transform is reversible and no information is lost in the process.

2. Filter the image using ideal lowpass and highpass filters with cutoff frequency

```python
# === Define a function to create ideal filters ===
def ideal_filter(shape, cutoff, highpass=False):
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2  # center
    mask = np.zeros((rows, cols), np.uint8)
    for u in range(rows):
        for v in range(cols):
            D = np.sqrt((u - crow)**2 + (v - ccol)**2) # calculate the distance to the center
            if highpass:
                mask[u, v] = 0 if D <= cutoff else 1 # if high pass filter, D<= cutoff is 0, else is 1
            else:
                mask[u, v] = 1 if D <= cutoff else 0 # low pass filter is inverse high pass filter
    return mask

# === Define filtering function ===
def apply_filter(img, cutoff, highpass=False):
    f = np.fft.fft2(img)
    fshift = np.fft.fftshift(f)

    # create filter mask
    mask = ideal_filter(img.shape, cutoff, highpass)
    fshift_filtered = fshift * mask # apply the filter

    # inverse transform
    f_ishift = np.fft.ifftshift(fshift_filtered)
    img_filtered = np.abs(np.fft.ifft2(f_ishift))
    return img_filtered, mask
```

Figure 3. Code for create low/high pass filter and apply it on the image

Step 1: Create a low-pass or high-pass filter by calculating the distance between every pixel and the center of the frequency-domain image (after applying the Fourier Transform). For a low-pass filter, set pixels whose distance from the center is less than the cutoff to 1 (keep low-frequency components), and others to 0. For a high-pass filter, do the opposite — set pixels within the cutoff distance to 0 and others to 1.

Step 2: Apply the mask to the Fourier-transformed image by multiplying them together, since the mask only contains binary values (0 or 1). Finally, apply the Inverse Fourier Transform to reconstruct the filtered image in the spatial domain.

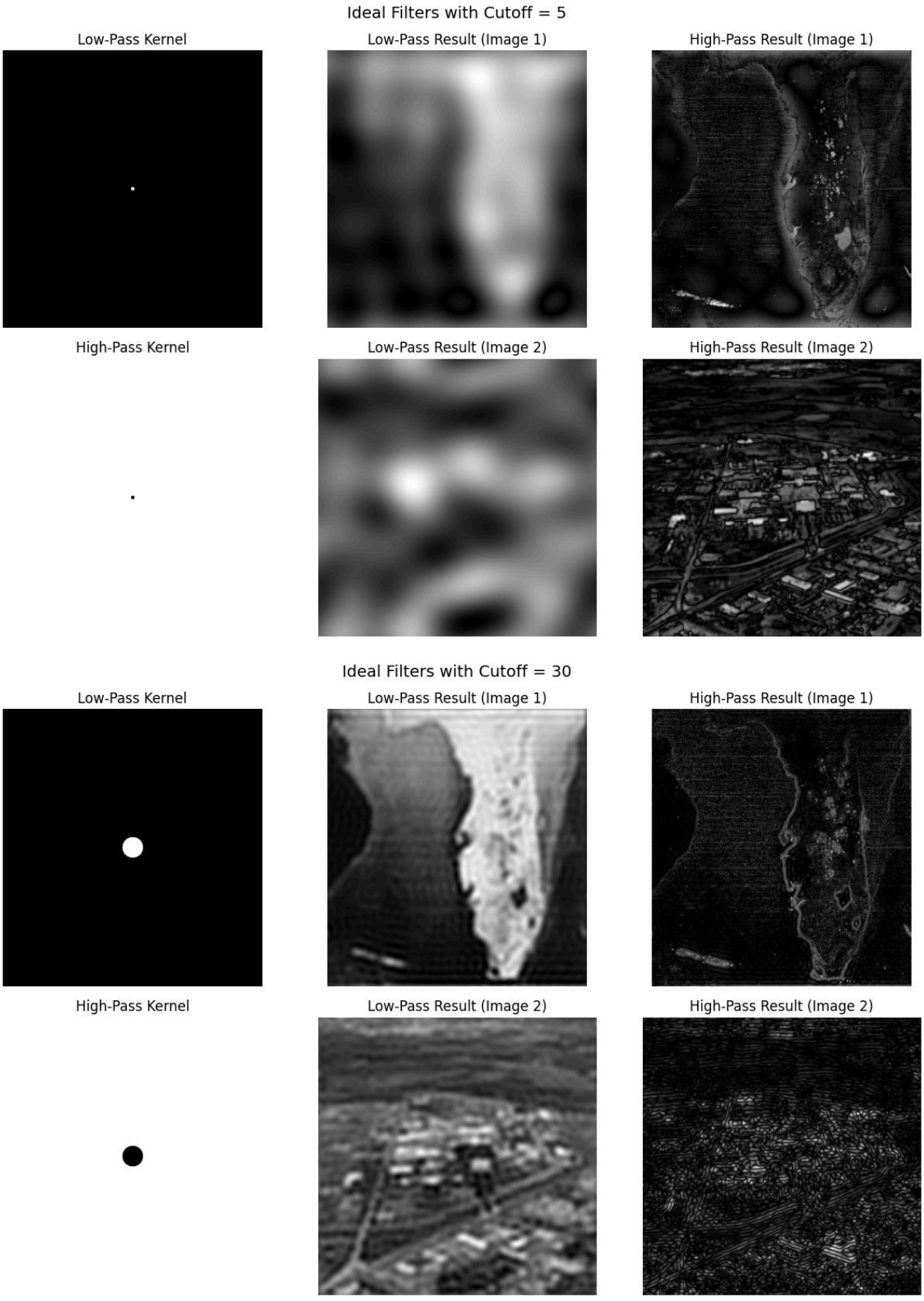


Figure 4, 5. The image after apply low/high pass filter with cutoff=5, 30

From Figures 4 and 5, several observations can be made. When applying a low-pass filter to the image, the high-frequency components are removed while the low-frequency components are preserved. As a result, the image appears blurred, because the high-frequency details — which correspond to sharp edges and fine textures — are filtered out.
When the cutoff frequency increases, more of the original frequency components are retained, so the filtered image looks closer to the original and becomes less blurred. On the other hand, when applying a high-pass filter, only the high-frequency components are preserved. This highlights regions with large intensity changes, which correspond to edges in the spatial domain. Therefore, high-pass filtering is commonly used for edge detection.
As the cutoff frequency increases, a larger portion of the low-frequency region is removed, meaning that less original information is preserved and the resulting image becomes more focused on edges and loses more overall brightness and texture.